

Customer Support AI Agent: Technical Audit & Final Report

Candidate Submission: SDE Intern Take-Home Assignment

Target Brand: @AmazonHelp

Dataset Source: Twitter Customer Support (TWCS) Dataset

1. Problem & Scope

1.1 Problem Statement

E-commerce customer service teams handle massive volumes of incoming social media requests. Manual resolution for every message creates high operational costs and slow response times. However, fully automated end-to-end resolution risks generating hallucinations, invalid policy claims, or unauthorized action commitments.

This project designs and evaluates an automated, grounded customer support AI agent for @AmazonHelp. The system combines intent classification, dense semantic retrieval, confidence-based escalation safeguards, and grounded reply generation to auto-handle routine inquiries while escalating ambiguous or unsupported requests to human agents.

1.2 Intent Taxonomy & Scope

The system defines a 7-intent classification schema:

Intent Label	Description	Scope Status
delivery_status	Package tracking, location updates, or delivery progress inquiries	Supported
delivery_delay_or_missing	Overdue, late, lost, or missing package reports	Supported
return_request	Queries regarding item returns or return procedures	Supported
refund_pending	Inquiries regarding pending, delayed, or missing refunds	Supported
wrong_or_damaged_item	Reports of defective, broken, wrong, or missing items in box	Supported
order_change_or_cancellation	Explicit requests to alter or cancel an order	Supported
out_of_scope	Non-support topics, device/digital issues, or general comments	Out of Scope

1.3 Out-of-Scope Boundary Definitions

1. **Rule Engine:** Scans text for explicit keyword patterns ("alexa", "prime" → out_of_scope; "return" → return_request; "refund" → refund_pending; "damaged item" → wrong_or_damaged_item; "cancel my order" → order_change_or_cancellation).
2. **LLM Fallback:** If no rule triggers, calls local Ollama LLM (qwen2.5:1.5b, temperature=0) with category guidelines and 150 reference examples (eval/intent_reference_examples.csv). Output is validated and defaults to out_of_scope if invalid.

3.2 Dense Semantic Retrieval (src/retrieve_embeddings.py)

- **Model:** SentenceTransformer("all-MiniLM-L6-v2") generating 384-dimensional dense vectors.
- **Corpus Storage:** Precomputed embeddings for 124,408 English customer messages stored as .npy array (amazonhelp_embeddings.npy / retrieval_embeddings.npy).
- **Query Execution:** Computes dot product matrix multiplication (CASE_EMBEDDINGS @ query_embedding) against normalized vectors to return top k=3 historical support pairs.

3.3 Escalation Decision Engine (src/escalate.py)

Determines operational routing using a deterministic 4-step check:

1. If intent == "out_of_scope" → escalate
2. If not retrieved_cases → escalate
3. If top_similarity < 0.60 → escalate
4. Otherwise → auto_handle

3.4 Grounded Reply Generation (src/generate_reply.py)

- **Model:** Local Ollama qwen2.5:1.5b (temperature=0).
- **Grounding & Privacy Constraints:** Embeds top-3 historical brand responses as grounding evidence. Historical customer messages are explicitly excluded from the prompt to prevent leaking third-party handles, names, or order numbers. Strictly forbids inventing tracking numbers, URLs, dates, or claiming unperformed actions.

4. Evaluation & Results

4.1 Quantitative Evaluation Summary

Benchmark / Metric	Evaluated Dataset	Score / Result	Benchmark Comparison / Note
Hybrid Intent Classifier Accuracy	eval/golden_test.csv (n=50)	50.00% (25/50)	Does NOT beat majority baseline
Majority-Class Baseline Accuracy	eval/golden_test.csv (n=50)	52.00% (26/50)	Predicts majority class (out_of_scope)
Human Review: Relevance	Human Subset (n=10)	100.00% (10/10)	Positive indicator rate (single rater)
Human Review: Helpfulness	Human Subset (n=10)	70.00% (7/10)	Positive indicator rate (single rater)

Benchmark / Metric	Evaluated Dataset	Score / Result	Benchmark Comparison / Note
Human Review: Grounding	Human Subset (n=10)	30.00% (3/10)	Positive indicator rate (single rater)

4.2 Key Evaluation Findings

- 1. **Classifier vs Baseline:** The hybrid intent classifier achieved **50.00%** accuracy on the 50-case held-out test set, failing to beat the **52.00%** majority-class baseline. Classification errors stemmed primarily from fine-grained semantic overlap between `delivery_status` and `delivery_delay_or_missing`.
- 2. **Human Evaluation Interpretation:** Human review across 10 non-out-of-scope test cases (`eval/human_agreement.csv`) yielded 100% relevance, 70% helpfulness, but only **30% grounding**. These figures represent positive indicator rates from **one human rater on n=10 cases**, NOT inter-rater agreement scores.
- 3. **Test Set Limitations:** The held-out test split contains exactly 50 cases and contains **zero representations** of the `order_change_or_cancellation` intent class.

5. Top 5 Real Failure Cases Analysis

Analysis of misclassified cases from `eval/intent_predictions.csv` (documented in `src/TOP_FIVE_FAILURES.md`):

■ Case ID	■ Customer Message Excerpt	■ Expected Intent	■ Predicted Intent	■
■ Case 18	■ "I'm unable to place deliveries to my new address..."	■ out_of_scope	■ delivery_status	■
■ Case 55	■ "No delivery and No refund!"	■ delivery_delay_or_missing	■ refund_pending	■
■ Case 59	■ "...hasn't even been despatched yet!"	■ return_request	■ delivery_status	■
■ Case 60	■ "An item was missing from a multi-order box..."	■ wrong_or_damaged_item	■ delivery_delay...	■
■ Case 72	■ "...deliveries ... returned to vendor..."	■ delivery_delay_or_missing	■ return_request	■

- 1. **Case 18 (Out-of-Scope → Delivery Status):** Customer reported an address-blocking error during checkout. The model over-indexed on the token *"deliveries"* and misclassified the account/address issue as a package tracking query.
- 2. **Case 55 (Delivery Delay → Refund Pending):** Customer message contained competing issue signals ("No delivery and No refund!"). The model prioritized the refund signal over the primary delivery failure.
- 3. **Case 59 (Return Request → Delivery Status):** Customer complained about pre-order dispatch delays in the context of a return. The phrase *"hasn't even been despatched yet"* led the model to predict `delivery_status`.
- 4. **Case 60 (Wrong/Damaged Item → Delivery Delay):** Customer reported an item missing from inside a delivered multi-item box. The model interpreted *"missing item"* as a missing delivery shipment rather than a missing package content item.
- 5. **Case 72 (Delivery Delay → Return Request):** Customer reported carrier delivery failures where packages were *"returned to vendor"*. The keyword phrase *"returned to vendor"* triggered `return_request` classification.

6. Misleading Headline Number & Unreliable Experiments

6.1 The "100% Relevance" Headline Trap

Reporting a **100% relevance score** based on the 10-case human subset as proof of high system quality is highly misleading:

- **Sample Bias:** The sample size is extremely small (n=10) and excludes out_of_scope queries.
- **Single Rater:** The score reflects a single subjective evaluator rather than cross-validated inter-rater agreement.
- **Overlooking Grounding:** While 100% of replies were topically relevant, only **30% were grounded**, meaning 70% of replies contained hallucinated details or unsupported assumptions.

6.2 Unreliable LLM-as-Judge Experiments

Two automated LLM-as-judge evaluation experiments were conducted using local qwen2.5:1.5b:

1. **LLM Reply Judge (src/evaluate_replies.py):** Outputted flat scores of 1 across relevance, helpfulness, grounding, and overall quality due to formatting/parsing failures.
2. **LLM Evidence Judge (src/evaluate_evidence.py):** Produced a **100.00% evidence support rate** by falsely labeling hallucinated short links (e.g. <https://t.co/...>) and unsupported statements as "supported".

Conclusion: The LLM evidence-judge score is an **unreliable, abandoned experimental artifact** and **MUST NOT** be cited as a valid system performance metric.

7. Next Steps — One Additional Week

If granted one additional week, engineering priorities would focus on:

1. **Classifier Fine-Tuning:** Fine-tune a specialized sequence classification model (e.g. DeBERTa-v3-base) on labelled e-commerce support data to resolve intent boundary confusion and comfortably beat the 52% baseline.
2. **Post-Generation Guardrails:** Implement strict regex and link-verifier output filters to strip hallucinated short links (<https://t.co/...>) and unauthorized commitments before presenting replies to users.
3. **Production Vector Database:** Migrate embedding storage from in-memory NumPy matrix multiplication to a dedicated vector index (e.g. FAISS or ChromaDB) with HNSW indexing for scalable sub-millisecond retrieval.
4. **Expanded & Balanced Evaluation Dataset:** Re-annotate a larger golden test set (n ≥ 500) ensuring balanced class representation across all 7 intents (specifically addressing the 0-case test sample for order_change_or_cancellation).
5. **Multi-Turn Context Support:** Extend dataset pairing beyond single tweet pairs to incorporate full multi-turn conversation thread history using build_threads.py.

8. Conclusion

This project successfully establishes an end-to-end e-commerce customer support pipeline for @AmazonHelp, incorporating data filtering over 124,408 English customer-agent pairs, dense semantic retrieval, deterministic escalation safeguards (threshold 0.60), and grounded LLM reply generation.

Rigorous evaluation demonstrates that while dense retrieval effectively surfaced relevant resolutions, small model capacity (qwen2.5:1.5b) led to low reply grounding (30%) and an intent classification accuracy (50.00%) that did not beat the majority baseline (52.00%). Automated LLM judges proved unreliable, reinforcing the critical necessity of manual human review and transparent benchmark reporting. The system serves as a solid research prototype but is

not production-ready.

Visual Module Flows

The following diagrams are added for readability and show the implemented module flow without changing the report's existing findings.

1. Data Preparation Flow

Raw TWCS Dataset — twcs.csv



Filter AmazonHelp brand replies



Match parent tweet IDs → Customer → AmazonHelp pairs



Language detection → English-only pairs



124,408 clean English pairs

2. Intent Classification Flow

Customer message



Rule engine checks explicit patterns



Rule match → supported intent / out_of_scope



No rule match → Qwen2.5 1.5B LLM



Validate output against 7 allowed labels



Final intent label

3. Retrieval Flow

Customer message



all-MiniLM-L6-v2 → 384-dimensional query embedding



Compare with normalized historical embeddings

Cosine-equivalent dot-product similarity



Top 3 historical support cases



4. Reply Generation Flow

Customer message + top-3 retrieved cases



Extract historical AmazonHelp responses



Build grounding-constrained prompt



Qwen2.5 1.5B (temperature = 0)



Concise AmazonHelp support reply

5. Escalation & End-to-End Flow

Customer message → Intent classification → Top-3 retrieval



out_of_scope? → Escalate



No retrieved cases? → Escalate



Top similarity < 0.60? → Escalate



Otherwise → Generate grounded reply → Auto-handle